جلسهٔ ۱۱ — هوموگرافی و تصحیحِ پرسپکتیو (Homography & Perspective Warp)

جزوهٔ درس — هفتهٔ یازدهم

نگاهِ کلی به این جلسه

تا این هفته یاد گرفتیم مقدارِ پیکسل‌ها را تغییر دهیم (روشنایی، لبه، آستانه) و نقاطِ متناظر را بینِ دو تصویر پیدا کنیم (ویژگی‌ها). این هفته کارِ متفاوتی می‌کنیم: موقعیتِ پیکسل‌ها را تغییر می‌دهیم. هدفِ نهایی بسیار ملموس است: کاربر با موبایل از یک کنتور یا سند عکسی کج و از زاویه می‌گیرد، و ما می‌خواهیم آن را به یک اسکنِ صاف و روبه‌رو تبدیل کنیم — درست مثلِ یک اسکنرِ واقعی. این کار قلبِ پروژهٔ DocScan است و ابزارِ ریاضیِ آن، **هوموگرافی** (homography) نام دارد.

یادداشت

چرا این مهم است؟ چون در هفته‌های ۱۲ تا ۱۴ می‌خواهیم ارقامِ کنتور را بخوانیم. خواندنِ رقم روی تصویری که صاف و روبه‌رو شده صدها بار آسان‌تر و دقیق‌تر است تا روی عکسی کج و پرسپکتیودار. پس این هفته، «مرحلهٔ آماده‌سازیِ» کلِ سامانه است.

یک تشبیهِ ساده

تصور کنید یک تابلوی مستطیلی روی دیوار است و شما از کنار به آن نگاه می‌کنید. در چشمِ شما، تابلو دیگر مستطیل نیست، بلکه یک ذوزنقه است: لبهٔ دور کوچک‌تر از لبهٔ نزدیک دیده می‌شود. مغزِ شما به‌طورِ خودکار می‌داند که تابلو «واقعاً» مستطیل است و آن را تصحیح می‌کند. هوموگرافی دقیقاً همین کارِ مغز را با ریاضی انجام می‌دهد: می‌داند ذوزنقهٔ دیده‌شده باید به مستطیل بازگردد و ماتریسی می‌سازد که این بازگشت را انجام دهد. کلِ این جلسه، آموزشِ همین «تصحیحِ زاویهٔ دید» به کامپیوتر است.

۱ تبدیلِ هندسی چیست؟

یک **تبدیلِ هندسی** (geometric transform) تابعی است که به هر پیکسلِ خروجی (x', y') می‌گوید مقدارش را از کدام محلِ تصویرِ ورودی (x, y) بردارد. عملیاتِ قبلی («این پیکسل چه رنگی است؟») را کنار می‌گذاریم و به پرسشِ تازه‌ای پاسخ می‌دهیم: «این نقطه کجا می‌رود؟»

- **انتقال (translation):** جابه‌جاییِ ساده.

- **چرخش و مقیاس (rotation، scale):** چرخاندن و بزرگ/کوچک کردن.
- **برش (shear):** کج‌کردنِ مستطیل به متوازی‌الاضلاع.
- **پرسپکتیو (perspective):** مهم‌ترین برای ما — اثرِ زاویهٔ دوربین.

۱.۱ سلسله‌مراتبِ تبدیل‌ها

تبدیل‌های دوبُعدی یک سلسله‌مراتب دارند؛ هر سطح عام‌تر از سطحِ بالای خود است و «درجاتِ آزادیِ» (DoF) بیشتری دارد:

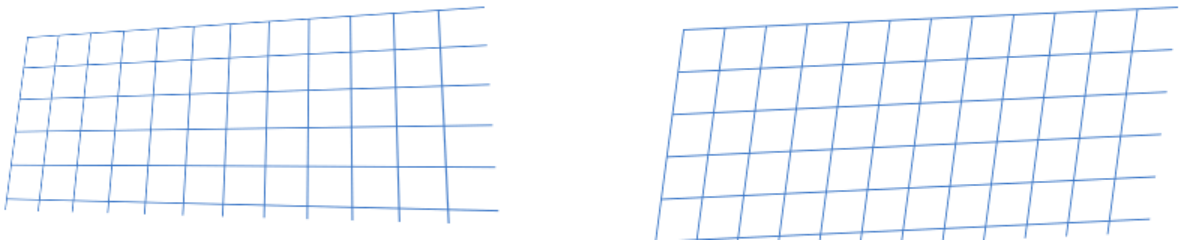
تبدیل	چه چیزی را حفظ می‌کند	DoF
صُلب (چرخش+انتقال)	طول و زاویه	۳
تشابهی (+مقیاس)	زاویه و نسبت‌ها	۴
اَفاین (Affine)	موازی‌بودنِ خطوط	۶
تصویری/هوموگرافی	فقط راستایِ خطوط	۸

نکتهٔ کلیدی

نکتهٔ کلیدی: یک عکس که از زاویه گرفته شده، به عام‌ترین تبدیل نیاز دارد، یعنی **هوموگرافی**. تبدیل‌های ساده‌تر (مثلِ اَفاین) نمی‌توانند اثرِ پرسپکتیو را مدل کنند.

۲.۱ اَفاین در برابرِ پرسپکتیو، روی یک شبکه

بهترین راه برای دیدنِ تفاوت، اعمالِ هر دو تبدیل روی یک شبکهٔ مربعی است:



چپ: اَفاین — خطوطِ موازی موازی می‌مانند راست: پرسپکتیو — خطوطِ موازی هم‌گرا می‌شوند

آن هم‌گراییِ خطوط در تصویرِ راست، همان اثرِ «نقطهٔ گریز» (vanishing point) در دوربینِ واقعی است. فقط هوموگرافی می‌تواند این را مدل کند.

۲ تبدیلِ اَفاین

تبدیلِ اَفاین ۶ درجهٔ آزادی دارد (یک ماتریسِ $\times$) و متوازی‌الاضلاع را به متوازی‌الاضلاع می‌نگارد.

ریاضیات

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = A \begin{pmatrix} x \\ y \end{pmatrix} + t, \qquad A \in \mathbb{R}^{۲\times۲},\ t \in \mathbb{R}^{۲}$$

خطوطِ صاف صاف می‌مانند و خطوطِ موازی موازی می‌مانند. با ۳ نقطهٔ متناظر کاملاً تعیین می‌شود: `cv2.getAffineTransform` و سپس `cv2.warpAffine`.

اَفاین برای چرخش/مقیاس/برش عالی است، اما نمی‌تواند صفحه‌ای را که از دوربین دور شده مدل کند — چون در آن حالت خطوطِ موازی باید هم‌گرا شوند و اَفاین این توانایی را ندارد.

۳ هوموگرافی

هوموگرافی یک ماتریسِ $\times$ است که دو نمای مختلف از یک سطحِ تخت را به هم مربوط می‌کند. ۸ درجهٔ آزادی دارد (درایهٔ نهم فقط یک عاملِ مقیاس است). خطوطِ صاف را به خطوطِ صاف می‌نگارد، اما خطوطِ موازی می‌توانند هم‌گرا شوند — دقیقاً مدلِ عکس‌گرفتن از یک صفحهٔ تخت از هر زاویه.

ریاضیات

با مختصاتِ همگن (homogeneous):

$$\begin{pmatrix} x' \\ y' \\ w' \end{pmatrix} = H \begin{pmatrix} x \\ y \\ ۱ \end{pmatrix}, \qquad H = \begin{pmatrix} h_{۱۱} & h_{۱۲} & h_{۱۳} \\ h_{۲۱} & h_{۲۲} & h_{۲۳} \\ h_{۳۱} & h_{۳۲} & ۱ \end{pmatrix}$$

و سپس پیکسلِ نهایی از تقسیم به دست می‌آید: $\left(x'/w',\ y'/w'\right)$.

۱.۳ چرا تقسیم بر w'؟ (بخشِ پرسپکتیو)

آن تقسیمِ نهایی همان چیزی است که اجسامِ دور را کوچک‌تر و خطوطِ موازی را هم‌گرا می‌کند:

- در اَفاین، سطرِ آخرِ ماتریس $(۰\ ۰\ ۱)$ است، پس همیشه $w' = ۱$ و هیچ پرسپکتیوی نداریم.
- در هوموگرافی، سطرِ آخر $(h_{۳۱}\ h_{۳۲}\ ۱)$ است، پس w' به موقعیتِ نقطه وابسته می‌شود و همین، اثرِ پرسپکتیو را می‌سازد.

نکتهٔ کلیدی

کلِ تفاوتِ اَفاین و پرسپکتیو در همین «تقسیم بر w'» غیرخطی است. اگر این تقسیم نباشد، هوموگرافی به اَفاین فرومی‌کاهد.

۲.۳ هوموگرافی چطور از نقاط حل می‌شود؟ (DLT)

شاید بپرسید `getPerspectiveTransform` پشتِ‌پرده چه می‌کند. الگوریتمِ آن **DLT** (Direct Linear Transform) نام دارد. برای هر تناظرِ $(x, y) \leftrightarrow (x', y')$ می‌دانیم:

ریاضیات

$$x' = \frac{h_{11}x + h_{12}y + h_{13}}{h_{31}x + h_{32}y + 1}, \qquad y' = \frac{h_{21}x + h_{22}y + h_{23}}{h_{31}x + h_{32}y + 1}$$

با ضربِ طرفین در مخرج، این دو رابطه به دو معادلهٔ خطی بر حسبِ درایه‌های H تبدیل می‌شوند. ۴ نقطه $\Rightarrow$ ۸ معادلهٔ خطی $\Rightarrow$ ۸ مجهول را یکتا حل می‌کند.

وقتی تعدادِ نقاط بیش از ۴ باشد (مثلِ ده‌ها تطبیق)، دستگاه «فرامعین» می‌شود و به‌جای حلِ دقیق، جوابِ کمینه‌مربعات را می‌گیریم (که `findHomography` با SVD انجام می‌دهد). این هم دقت را بالا می‌برد و هم اثرِ نویز را کم می‌کند.

نکتهٔ کلیدی

درسِ مهندسی: تقریباً همیشه بیش از حداقلِ داده جمع کنید. ۴ نقطه کافی است اما شکننده؛ ۴۰ نقطه با کمینه‌مربعات، برآوردی پایدار و مقاوم به نویز می‌دهد.

۳.۳ چند نقطه یک هوموگرافی را تعیین می‌کند؟

۸ مجهول داریم، پس به ۸ معادله نیاز است. هر تناظرِ نقطه‌ای ۲ معادله می‌دهد (یکی برای x' و یکی برای y')، پس **۴ تناظرِ نقطه‌ای** کافی است.

- برای دقیقاً ۴ نقطه: `cv2.getPerspectiveTransform(` , `)`.
- برای ده‌ها تطبیقِ نویزی: `cv2.findHomography(pts,` `pts, cv2.RANSAC)`.

یادداشت

۴ گوشهٔ خوب تمامِ چیزی است که نیاز دارید. در DocScan، این ۴ گوشه همان ۴ گوشهٔ کادرِ سند یا کنتور است — که در هفتهٔ ۹ با `approxPolyDP` یاد گرفتیم پیدا کنیم.

۴ تصحیح (Rectification): قلبِ DocScan

حالا همهٔ قطعات را کنار هم می‌گذاریم. فرآیندِ تبدیلِ عکسِ کج به اسکنِ صاف چهار گام دارد:

۱. ۴ گوشهٔ سند/کنتور را در عکس پیدا کن (با `approxPolyDP` از هفتهٔ ۹، یا با تطبیقِ ویژگی).

۲. تصمیم بگیر این گوشه‌ها کجا باید بروند: ۴ گوشهٔ یک مستطیلِ تمیز.

۳. هوموگرافیِ «گوشه‌های عکس ← گوشه‌های مستطیل» را حساب کن.

۴. عکس را با `warpPerspective` از این هوموگرافی عبور بده: خروجی، اسکنِ صاف و روبه‌روست.

۱.۴ گام ۱ و ۲: کنتورِ کج و ۴ گوشه‌اش

در شکلِ زیر، سمتِ چپ الگوی تمیز (هدف) است و سمتِ راست یک عکسِ کجِ شبیه‌سازی‌شده که ۴ گوشه‌اش با نقاطِ قرمز مشخص شده‌اند:





چپ: الگوی تمیز (مقصد) راست: عکسِ کج با ۴ گوشهٔ علامت‌گذاری‌شده

این ۴ گوشه همان تناظرها هستند: هر نقطهٔ قرمز کجاست در برابرِ اینکه کجا باید باشد.

۲.۴ گام ۳ و ۴: تبدیل به اسکنِ صاف

با محاسبهٔ هوموگرافی از گوشه‌های عکس به گوشه‌های مستطیل و اعمالِ `warpPerspective`، عکسِ کج به نمای صاف و روبه‌رو تبدیل می‌شود:





چپ: ورودیِ کج راست: خروجیِ صاف‌شده — ارقام حالا خوانا هستند

۳.۴ یک مثالِ عددی: نگاشتِ یک نقطه

بیایید ببینیم یک هوموگرافی روی یک نقطه چه می‌کند. فرض کنید

ریاضیات

$$H = \begin{pmatrix} ۱٫۲ & ۰٫۱ & ۵ \\ ۰٫۰۵ & ۱٫۱ & ۸ \\ ۰٫۰۰۰۵ & ۰٫۰۰۰۳ & ۱ \end{pmatrix}, \qquad \text{نقطهٔ ورودی } (x, y) = (۱۰۰, ۵۰)$$

گامِ ۱ — ضربِ ماتریسی:

$$x' = ۱٫۲(۱۰۰) + ۰٫۱(۵۰) + ۵ = ۱۳۰$$
$$y' = ۰٫۰۵(۱۰۰) + ۱٫۱(۵۰) + ۸ = ۶۸$$
$$w' = ۰٫۰۰۰۵(۱۰۰) + ۰٫۰۰۰۳(۵۰) + ۱ = ۱٫۰۶۵$$

گامِ ۲ — تقسیم بر w':

$$\left(\frac{۱۳۰}{۱٫۰۶۵}, \frac{۶۸}{۱٫۰۶۵}\right) \approx (۱۲۲٫۱,\ ۶۳٫۸)$$

توجه کنید که اگر سطرِ آخر $(۰, ۰, ۱)$ بود، $w' = ۱$ می‌شد و خروجی دقیقاً $(۱۳۰, ۶۸)$ — یعنی یک تبدیلِ اَفاین. آن درایه‌های کوچکِ ۰٫۰۰۰۵ و ۰٫۰۰۰۳ در سطرِ آخر، همان چیزی هستند که پرسپکتیو را می‌سازند و مقدار را کمی «فشرده» می‌کنند.

نکتهٔ کلیدی

همین محاسبهٔ ساده برای هر پیکسل تکرار می‌شود. زیباییِ کار این است که یک ماتریسِ کوچکِ × کلِ تغییرِ شکلِ پیچیدهٔ پرسپکتیو را کد می‌کند.

۴.۴ ترتیبِ درستِ گوشه‌ها

یک نکتهٔ عملیِ حیاتی: `approxPolyDP` چهار نقطه می‌دهد اما به ترتیبِ دلخواه. باید آن‌ها را مرتب کنیم: بالاچپ، بالاراست، پایین‌راست، پایین‌چپ — وگرنه تصویرِ خروجی می‌چرخد یا آینه می‌شود.

ریاضیات

یک ترفندِ ساده برای مرتب‌کردن: نقطه‌ای که $x+y$ کمینه دارد بالاچپ است؛ بیشینهٔ $x+y$ پایین‌راست؛ کمینهٔ $x-y$ بالاراست؛ بیشینهٔ $x-y$ پایین‌چپ.

هشدار / دام

ترتیبِ غلطِ گوشه‌ها = اسکنِ آینه‌ای یا چرخیده. همیشه گوشه‌ها را با یک قاعدهٔ ثابت مرتب کنید.

۵ انطباقِ خودکار با RANSAC

به‌جای انتخابِ دستیِ ۴ گوشه، می‌توانیم از تطبیق‌های ویژگیِ هفتهٔ ۱۰ استفاده کنیم. اما تطبیق‌ها شاملِ مواردِ غلط‌اند؛ یک برازشِ ساده با این برون‌داده‌ها خراب می‌شود. راه‌حل، **RANSAC** است.

نکتهٔ کلیدی

RANSAC مخففِ RANdom SAmple Consensus است: تبدیلی را پیدا می‌کند که بیشترین تعدادِ تطبیق‌ها با آن «موافق»‌اند (یعنی درون‌داده یا inlier هستند).

۱.۵ RANSAC گام‌به‌گام

۱. ۴ تناظرِ تصادفی انتخاب کن و یک H کاندید حساب کن.

۲. درون‌داده‌ها را بشمار: تطبیق‌هایی که H نقطه‌شان را با خطای چند پیکسل درست می‌نگارد.

۳. این کار را ده‌ها یا صدها بار تکرار کن و H با بیشترین درون‌داده را نگه دار.

۴. در پایان H را با همهٔ درون‌داده‌هایش دوباره برازش کن تا دقیق شود.

یادداشت

قدرتِ RANSAC: حتی اگر کسرِ بزرگی از تطبیق‌ها غلط باشند، تا وقتی که ۴ تطبیقِ درست وجود داشته باشند، RANSAC آن‌ها را پیدا می‌کند. به همین دلیل استانداردِ طلاییِ برازشِ مقاوم در بینایی ماشین است.

۲.۵ تصحیحِ خودکار روی کنتورِ واقعی

در پروژهٔ واقعیِ ما، این خط‌لولهٔ کامل روی یک کنتورِ واقعیِ آب اجرا شد: ابتدا ویژگی‌های ORB روی عکسِ کج و الگو استخراج شد، سپس تطبیق داده شد، و در نهایت `findHomography` با RANSAC تبدیل را برآورد کرد. نتیجه: از **۶۶ تطبیقِ خوب، ۵۷ درون‌داده** و یک اسکنِ کاملاً صاف — بدونِ هیچ انتخابِ دستیِ گوشه:





چپ: عکسِ کج راست: تصحیحِ خودکار با RANSAC + ORB (۵۷ درون‌داده از ۶۶ تطبیق)

۳.۵ نگاشتِ مستقیم در برابرِ نگاشتِ معکوس

یک ظرافتِ پیاده‌سازی: برای پُرکردنِ تصویرِ خروجی، دو راه داریم. در **نگاشتِ مستقیم** (forward) هر پیکسلِ ورودی را به خروجی می‌فرستیم؛ مشکل این است که برخی پیکسل‌های خروجی خالی می‌مانند و برخی دوبار پُر می‌شوند («سوراخ» و «برخورد»). در **نگاشتِ معکوس** (inverse) که OpenCV استفاده می‌کند، برای هر پیکسلِ خروجی با معکوسِ H برمی‌گردیم و می‌پرسیم مقدارش از کجای ورودی می‌آید — پس هیچ سوراخی نمی‌ماند.

یادداشت

به همین دلیل `warpPerspective` داخلی از H^{-1} استفاده می‌کند و در ترکیب با درون‌یابی، خروجیِ صاف و بدونِ حفره تولید می‌کند. شما فقط H را می‌دهید؛ OpenCV بقیه را انجام می‌دهد.

۶ ژرف‌نگری

۱.۶ درون‌یابی: پُرکردنِ پیکسل‌های خروجی

هنگامِ warp، هر پیکسلِ خروجی به یک محلِ کسری در تصویرِ ورودی نگاشته می‌شود — جایی بینِ پیکسل‌ها. باید مقدار را «درون‌یابی» (interpolate) کنیم:

- **نزدیک‌ترین همسایه (nearest):** سریع اما پله‌ای/دندانه‌دار.
- **دوخطی (bilinear):** نرم، پیش‌فرضِ خوب برای تصویر (`INTER_LINEAR`).
- **دومکعبی (bicubic):** تیزتر اما کندتر.

نکتهٔ کلیدی

برای تصویرهای معمولی از `INTER_LINEAR` استفاده کنید؛ اما برای نقشه‌های برچسب/ماسک (که مقدارشان معنای رده دارد) از `INTER_NEAREST`، وگرنه بینِ دو برچسب مقدارِ بی‌معنی ساخته می‌شود.

۲.۶ فرضِ تخت‌بودن و محدودیت‌هایش

هوموگرافی فقط برای یک سطحِ تخت (یا دوربینی که فقط می‌چرخد) دقیق است. یک صفحهٔ خمیده، یک درپوشِ کنتورِ قوس‌دار، یا اجسامِ سه‌بُعدی این مدل را می‌شکنند.

هشدار / دام

اگر «اسکنِ» شما هنوز در لبه‌ها قوس دارد، سطح تخت نبوده است. یک هوموگرافیِ واحد نمی‌تواند انحنا را تصحیح کند؛ برای صفحهٔ خمیده به «dewarping» (شبکهٔ غیرخطی) نیاز است که فراتر از یک H است.

۳.۶ دو مسیرِ یافتنِ گوشه‌ها: کدام را انتخاب کنیم؟

در عمل دو راه برای رسیدن به تناظرها داریم و انتخاب به شرایط بستگی دارد:

- **مسیرِ کانتور (هفتهٔ ۹):** لبهٔ بیرونیِ سند را پیدا کن، با `approxPolyDP` به ۴ گوشه تقریب بزن. سریع و ساده است، اما نیازمندِ آن است که کلِ سند در کادر و لبه‌اش واضح باشد. برای اسکنِ یک برگه روی میزِ تیره عالی است.
- **مسیرِ ویژگی (هفتهٔ ۱۰):** ویژگی‌ها را با یک الگو تطبیق بده و با RANSAC هوموگرافی بگیر. وقتی لبهٔ سند نامشخص است اما بافتِ داخلی غنی است (مثلِ کنتور با ارقام و علائم) بهتر کار می‌کند، و حتی اگر بخشی از سند بیرونِ کادر باشد باز هم جواب می‌دهد.

نکتهٔ کلیدی

در DocScan ما، مسیرِ ویژگی + RANSAC انتخابِ اصلی است، چون کنتور بافتِ داخلیِ غنی دارد و همیشه لبهٔ تمیزی در دسترس نیست. اما داشتنِ هر دو مسیر به‌عنوانِ پشتیبان، سامانه را مقاوم‌تر می‌کند.

۴.۶ شکست‌های رایج در DocScan و درمانشان

- **اسکنِ آینه/چرخیده:** گوشه‌ها بد مرتب شده‌اند $\rightarrow$ قاعدهٔ مرتب‌سازیِ $x \pm y$ را اعمال کنید.
- **کششِ عجیبِ ارقام:** نسبتِ ابعادِ مستطیلِ مقصد اشتباه است $\rightarrow$ نسبتِ واقعیِ کنتور را اندازه بگیرید.
- **لبه‌ها هنوز قوس دارند:** سطح تخت نبوده (درپوشِ کنتور خمیده) $\rightarrow$ به dewarping غیرخطی نیاز است.
- **هوموگرافیِ بی‌معنی:** تعدادِ درون‌داده‌ها بسیار کم است $\rightarrow$ ویژگیِ بیشتری استخراج کنید یا آستانهٔ RANSAC را تنظیم کنید؛ اگر باز هم کم بود، احتمالاً عکس و الگو واقعاً یکی نیستند.

هشدار / دام

همیشه تعدادِ درون‌داده‌های RANSAC را چاپ و بررسی کنید. اگر از یک آستانه (مثلاً ۱۵) کمتر بود، به هوموگرافی اعتماد نکنید و به کاربر پیام دهید که دوباره عکس بگیرد — این یک «نردبانِ اطمینان» در سامانهٔ واقعی است.

۵.۶ کاربردهای دیگرِ هوموگرافی

- **دوختِ پانوراما:** هر عکس را به یک قابِ مشترک warp کن و ادغام کن.
- **واقعیتِ افزوده:** یک گرافیک را روی یک نشانگرِ تختِ شناسایی‌شده قرار بده.
- **نمای پرنده (bird's-eye):** صحنهٔ جاده/پارکینگ را به نقشهٔ بالابه‌پایین تصحیح کن.
- **DocScan:** تصحیح‌گرِ کنتور/پلاکِ ما.

۷ کدِ کاملِ تصحیح‌گر

```
import cv2, numpy as np
# corners: 4 page corners found in the photo, ordered TL,TR,BR,BL
W, H = 800, int(800 * (414.1          # A4-ish output
dst = np.float32([[0,0],[W,0],[W,H],[0,H]])
M = cv2.getPerspectiveTransform(corners, dst)
scan = cv2.warpPerspective(photo, M, (W, H))

# automatic variant (no manual corners): use ORB matches
Hr, mask = cv2.findHomography(pts_photo, pts_template, cv2.RANSAC, (0.5
scan = cv2.warpPerspective(photo, Hr, (W, H))
print("inliers:", int(mask.sum()))
```

چهار خط، یک عکسِ کج را به یک اسکنِ صاف تبدیل می‌کند — قلبِ DocScan.

۸ تمرینِ کلاسی و پاسخ

تمرین

۱) چرا یک تبدیلِ آفاین نمی‌تواند صفحه‌ای را که از زاویه عکس گرفته شده صاف کند؟

۲) ۴ گوشهٔ سند را در عکس دارید. برای یک اسکنِ به‌نسبتِ A۴ با عرضِ ۸۰۰، نقاطِ مقصد چه هستند؟

۳) خط‌لولهٔ خودکارِ شما ۲۰۰ تطبیق دارد که ۶۰تای آن‌ها غلط‌اند. کدام گام نجاتتان می‌دهد؟

پاسخ‌ها: (۱) آفاین خطوطِ موازی را موازی نگه می‌دارد، اما عکسِ واقعی آن‌ها را هم‌گرا می‌کند؛ این نیازمندِ «تقسیم بر w'» است که فقط هوموگرافی دارد. (۲) نقاطِ مقصد $(۰,۰)$, $(۸۰۰,۰)$, $(۸۰۰,۱۱۳۱)$, $(۰,۱۱۳۱)$؛ چون نسبتِ A۴ حدودِ : است، ارتفاع $۸۰۰ \times ۱/۴۱۴ \approx ۱۱۳۱$. (۳) **RANSAC** درونِ `findHomography` — ۶۰ برون‌داده را نادیده می‌گیرد و فقط به درون‌داده‌ها برازش می‌دهد.

۹ چطور کیفیتِ تصحیح را بسنجیم؟

مثلِ هفتهٔ پیش، به یک معیارِ کمی نیاز داریم تا بدانیم تصحیح خوب بوده یا نه. سه سنجهٔ عملی:

- **تعدادِ درون‌داده‌های RANSAC:** هرچه بیشتر، برازش پایدارتر. زیرِ یک آستانه یعنی بی‌اعتماد.
- **خطای بازتصویر (reprojection error):** پس از یافتنِ H، نقاطِ مبدأ را با آن بنگارید و فاصله‌شان با نقاطِ مقصدِ واقعی را بسنجید؛ میانگینِ چندپیکسلی خوب است.
- **راست‌بودنِ خطوط:** در اسکنِ نهایی، لبه‌های کادرِ کنتور باید افقی/عمودی باشند؛ اگر کج‌اند، تصحیح ناقص است.

ریاضیات

خطای بازتصویر برای یک نقطه: $e_i = \| \text{مقصدِ واقعی}_i - H \cdot \text{مبدأ}_i \|$. معیارِ کل معمولاً ریشهٔ میانگینِ مربعات (RMSE) روی همهٔ درون‌داده‌هاست. این عدد، «کیفیتِ هندسیِ» اسکن را به‌صورتِ یک کمیتِ قابلِ‌گزارش می‌دهد.

یادداشت

در سامانهٔ واقعی، این اعداد را در لاگ ثبت کنید. وقتی نرخِ خطای خواندنِ ارقام (هفته‌های بعد) بالا رفت، اولین جای بررسی این است: آیا تصحیحِ پرسپکتیو ضعیف بوده؟ زنجیرهٔ پردازش را باید بتوان مرحله‌به‌مرحله عیب‌یابی کرد.

۱۰ جمع‌بندی

در این جلسه یاد گرفتیم که تبدیل‌های هندسی موقعیتِ پیکسل‌ها را تغییر می‌دهند و در یک سلسله‌مراتب (صُلب $\rightarrow$ اَفاین $\rightarrow$ تصویری) قرار می‌گیرند؛ هوموگرافی عام‌ترین تبدیلِ یک سطحِ تخت است و با ۴ تناظر تعیین می‌شود؛ با ترتیب‌دادنِ گوشه‌ها، ساختنِ H و `warpPerspective` می‌توان یک عکسِ کج را به اسکنِ صاف تبدیل کرد؛ و با `findHomography` به‌همراهِ RANSAC می‌توان این کار را به‌طورِ خودکار از تطبیق‌های ویژگی انجام داد. این، مرحلهٔ آماده‌سازیِ DocScan است: از هفتهٔ آینده روی همین اسکن‌های صاف، خواندنِ ارقام را با یادگیریِ ماشین و شبکه‌های عصبی آغاز می‌کنیم.